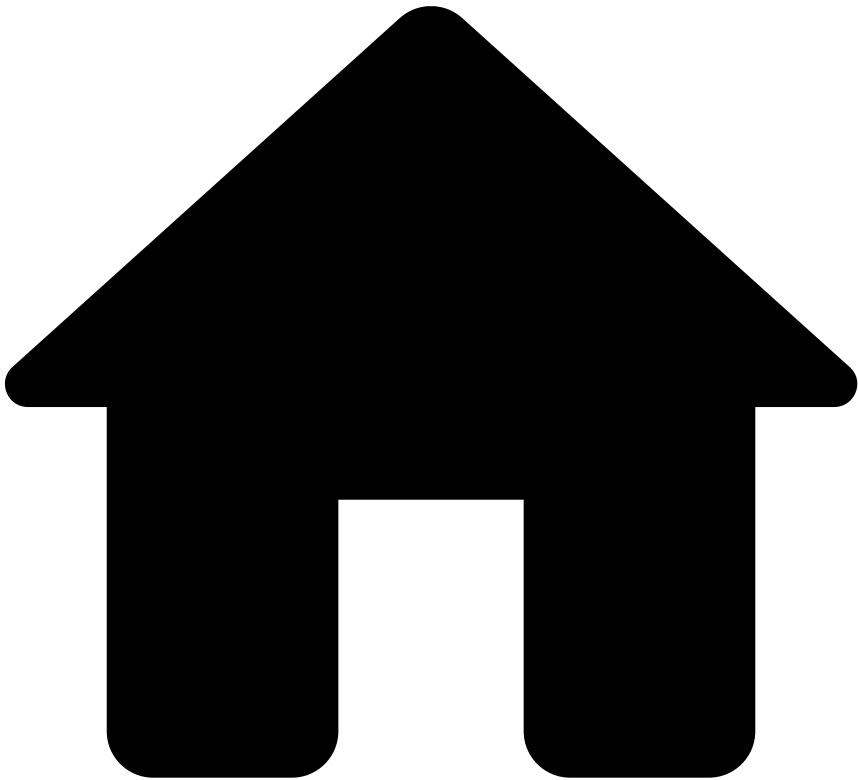


-



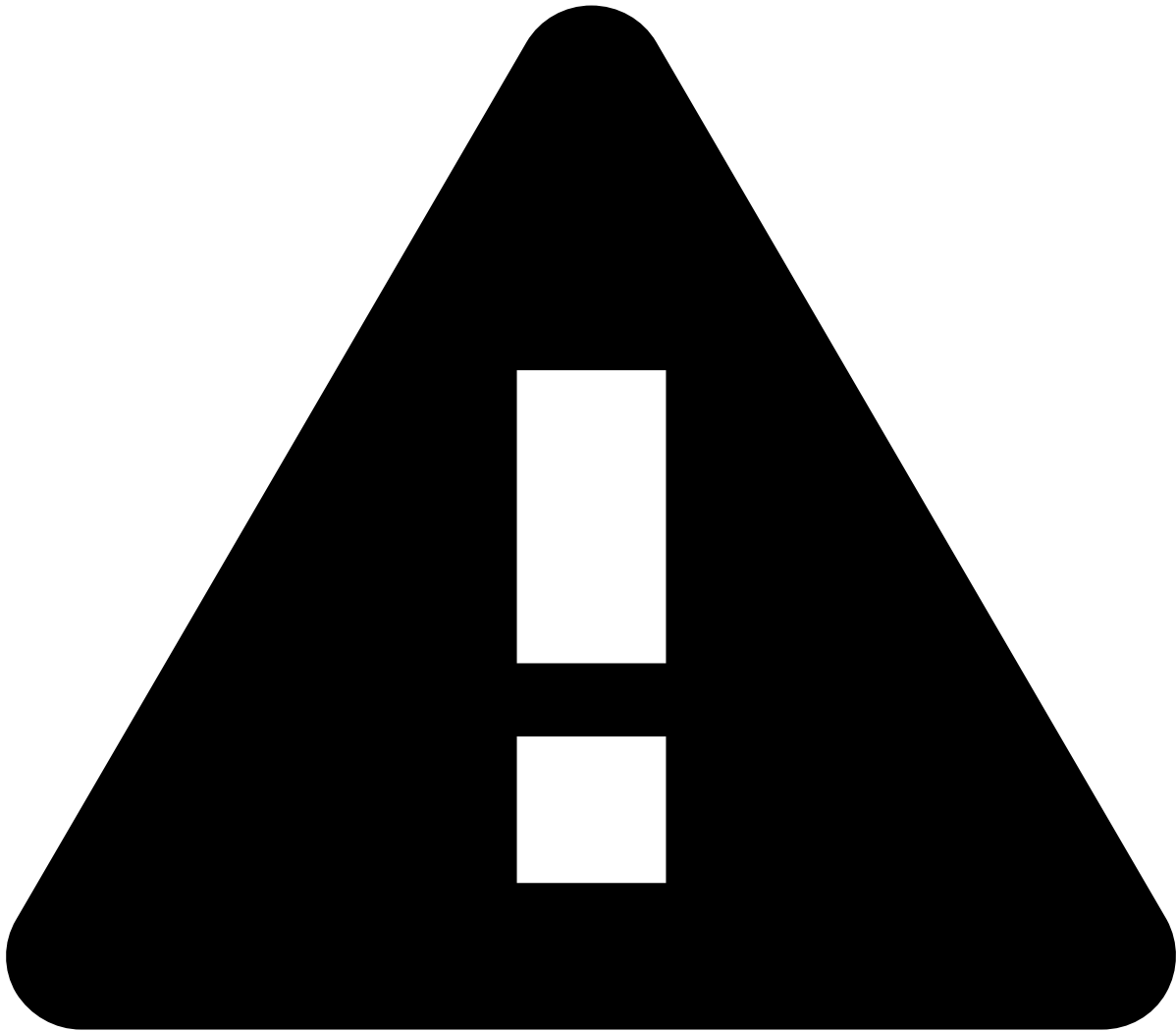
- [Rule Guide](#)
- Rules for Advanced Users

On this page

Was this page helpful? 

Rules for Advanced Users

This page provides advanced examples of using Pulse to write [Rules](#), via a selection of use cases.



The Examples are not Production Code

Note that the examples were simplified to better showcase the concepts and constructs of RDL. The code available in this page may not cover all the cases of each scenario and should not be used without review in Production environments.

Access to Historical Data🔗

In this example we will see a use-case where we need to access historical data. Let us say we want to detect three consecutive transactions with an amount greater than \$1000 that occurred in the last two hours.

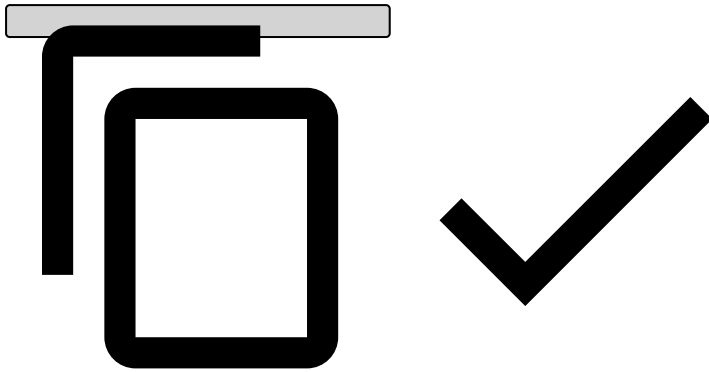
The first thing to do is to write some auxiliary variables. These will contain the main logic that will be used in the condition block.

Variables

```
// Collect all transactions in a two hours window
window2hours = event[every 3 tuples for 2 hours where amount >= 1000];

maybeFraud = match
  if (event.amount < 1000) then // This breaks the consecutive >= $1000 transactions.
    window2hours.clear();
    return false;
  else if (window2hours.size () == 3) then // We got 3 transactions >= $1000 within 2 hours.
    return true; // We got 3 transactions in our 2 hours window. Maybe that is fraud.
```

```
else
    return false; // Still no fraud.
```



We start by declaring a window where events will stay for 2 hours, since that is our time-span. Also, that will contain at most 3 events that specify the `amount >= 1000` predicate. Every other event will not be part of the window.

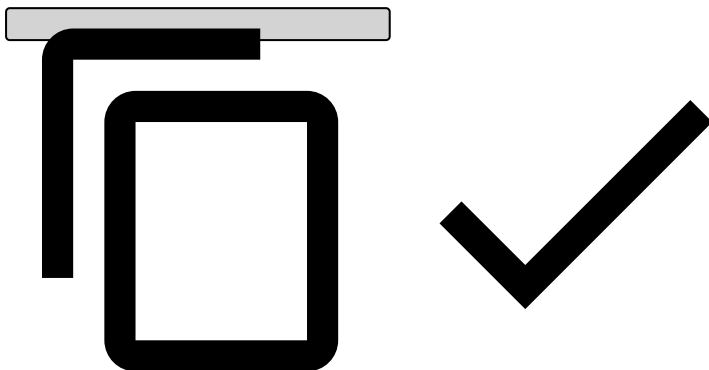
So, every time a new event arrives, it will either enter the window or not. Either way, we still have to decide what to do with this event. This leaves us with three possible scenarios:

1. **The event has an amount of less than \$1000** - This breaks our requirement of three consecutive transactions of more than \$1000. In this case, we must explicitly clear the window, because previous transactions of less than \$1000 can no longer be taken into consideration for our fraud pattern.
2. **The event has an amount greater or equal than \$1000 and the window already has three events** - This represents the fraud pattern we are trying to detect: three consecutive transactions over \$1000 over a two hours window.
3. **The event has an amount greater or equal then \$1000 and the window still has less than three events** - The current event contributes to our pattern, but it is still not the third consecutive transaction over \$1000.

On each incoming event, after we have gone through all possible scenarios, the `maybeFraud` variable will be **true** if the pattern has been found or **false** otherwise. The condition will be a simple check on this variable:

Condition

```
maybeFraud
```



Using Simple Lists to Group Data

The most common case for using a simple [List](#) is to maintain lists of items and then check for the existence of a specific item. For example, to maintain a list of blacklisted card numbers:

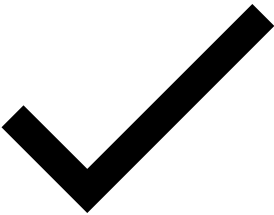
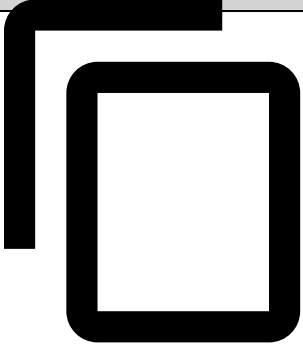
Figure 1. Creating a New Simple List for Blacklisted Card Numbers

Figure 2. Adding Items to the Card Blacklist

To check if a card is in a list write the condition as follows:

How to check if a card is in a List

```
card_blacklist.contains(event.card_number)
```





Explicitly iterating over Lists is not recommended

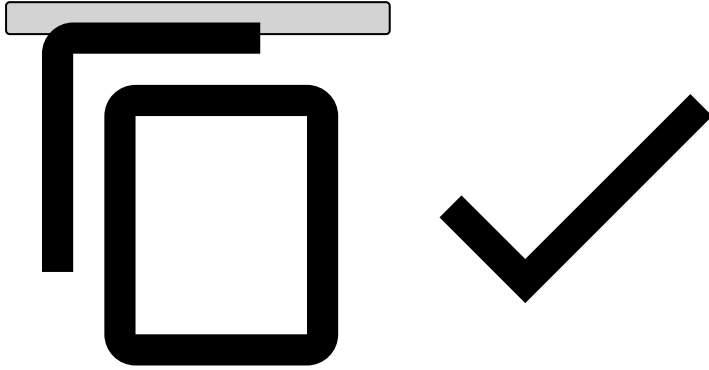
If you find yourself wanting to explicitly iterate over a List (e.g., using `filter`, `all`, etc) consider rewriting the rule, as this is highly inefficient as Lists are optimized for `contains`/lookup operations.

Using Complex Lists to Efficiently Store Common Patterns [🔗](#)

Imagine that it is necessary to block certain card numbers for certain countries. Suppose for example that we want to block 379867957393437 in USA and Canada and 340735948409065 in Spain and Italy. One possible approach would be to code this directly in RDL:

```
// Variables
matchCard3437 = event.cardnumber == "379867957393437" and
(event.country == "USA" or event.country == "CANADA");
matchCard9065 = event.cardnumber == "340735948409065" and
(event.country == "SPAIN" or event.country == "ITALY");

// Condition
matchCard3437 or matchCard9065
```



There are several problems with this approach:

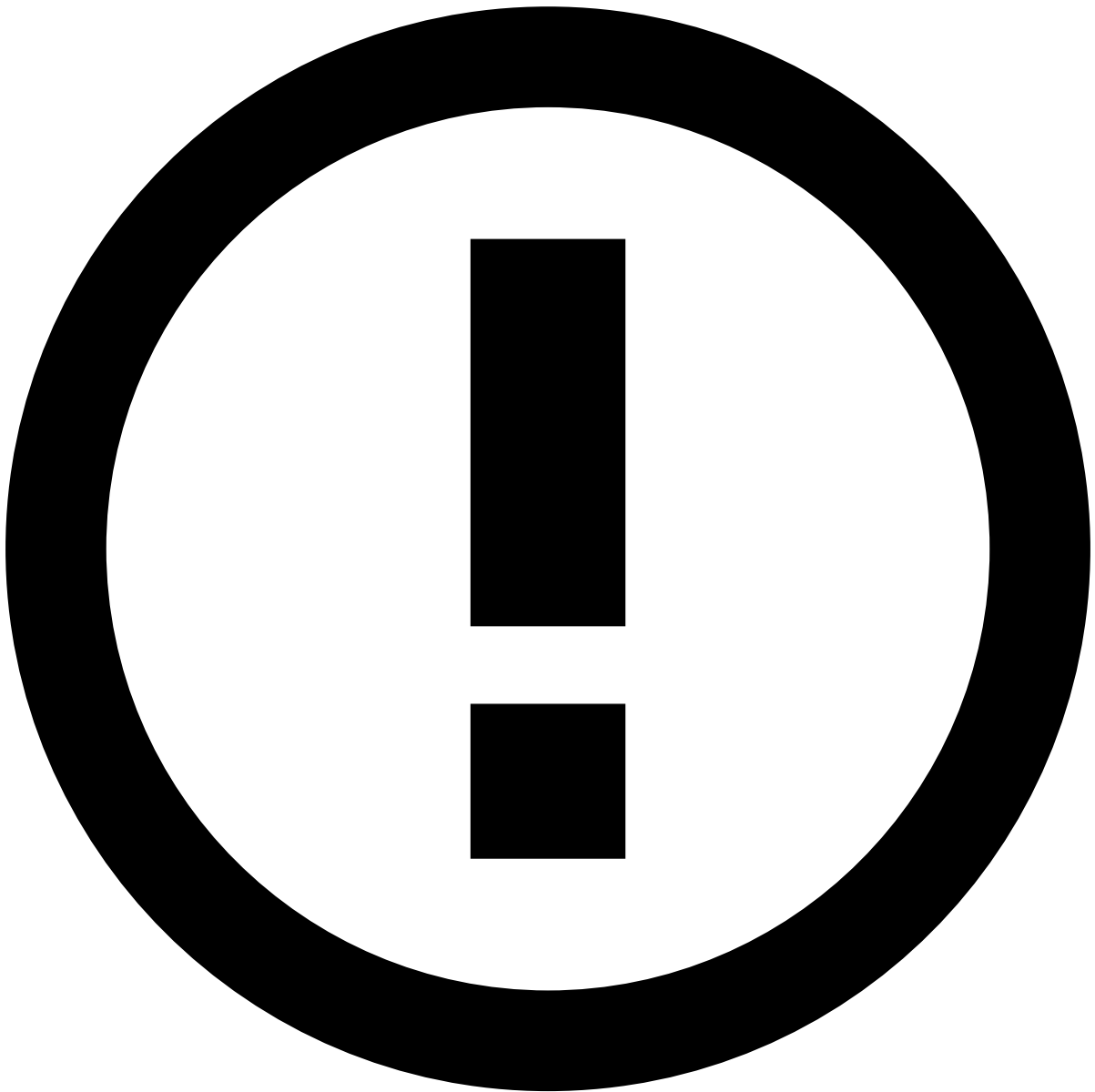
- It does not scale well as you need to add more card numbers and countries
- It makes the process of maintaining the rule error prone
- It does not allow to an Administrator to create a permission for a specific user to add or remove credit card numbers (and not change the rule logic)

The way to address these issues is to use the [Lists](#), which allow us to create a list with the card number to blocked country list mapping.

First we want a complex list that has the card number as the key and has a blocked countries list as a field:

Figure 3. Creating a Complex List to Store Card Number to Blocked Countries List Pairs

Note that we choose the type of the key as string. This is a good idea for credit card numbers because it allows you to have hashed card numbers as keys.



Choosing the Correct List Type (Simple or Complex)

Complex lists are a good choice when it is necessary to perform lookups because they are internally implemented as a Map. Simple lists on the other hand are more suited for contains and exists operations because they are internally implemented as a Set.

Next we need to create some items:

Figure 4. Adding an Item to a Complex List

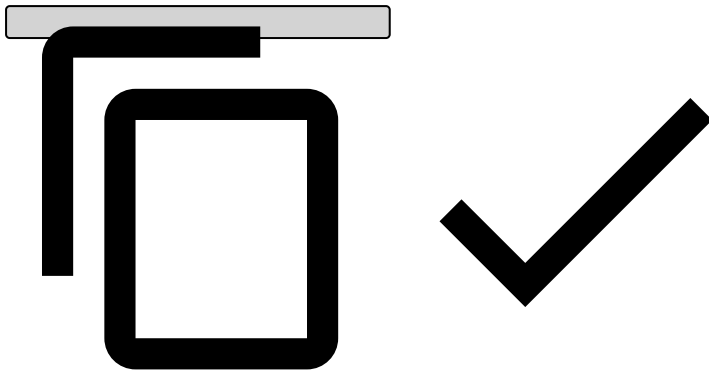
Figure 5. Adding Another Item to a Complex List

Once we got the list setup, we can use it to rewrite our rule, resulting in a much more simple and maintainable code:

```
// Variables
cardnum_entry = cardnumber__country[event.cardnumber];

// Condition
// We first need to check if there is any entry for the given card number
```

```
// and then we check if the blocked countries list contains the country
// where the current transaction originated from
cardnum_entry != null and cardnum_entry.blocked_countries.contains(event.country);
```



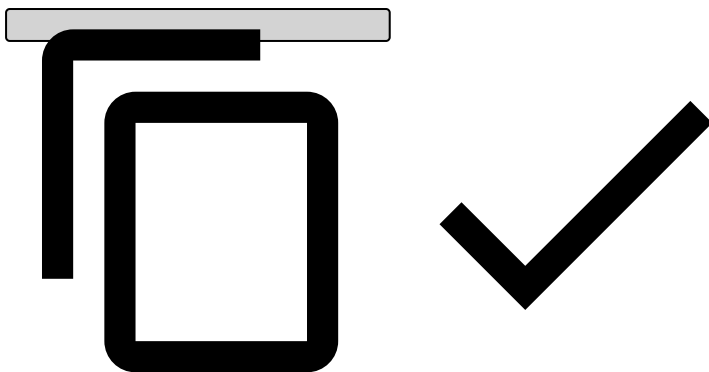
Correlating Multiple Transactions to Detect Known Fraud Patterns

To access past transactions a window can be used. In RDL, a window is a time ordered set of transactions that has a range, which is either numerical or temporal.

Example 3: match transactions of 1\$ followed by a transaction in a different location

```
// Variables
// Get the last 2 transactions for the current card number
last2Events = event[2 tuples by card_number];
previous = last2Events.at(1); // Collections in RDL start in index 1

// Condition
// The count check is necessary because the window may have 0, 1 or 2 events
(last2Events.count() == 2) and (previous.amount <= 1) and (previous.locations != event.location)
```



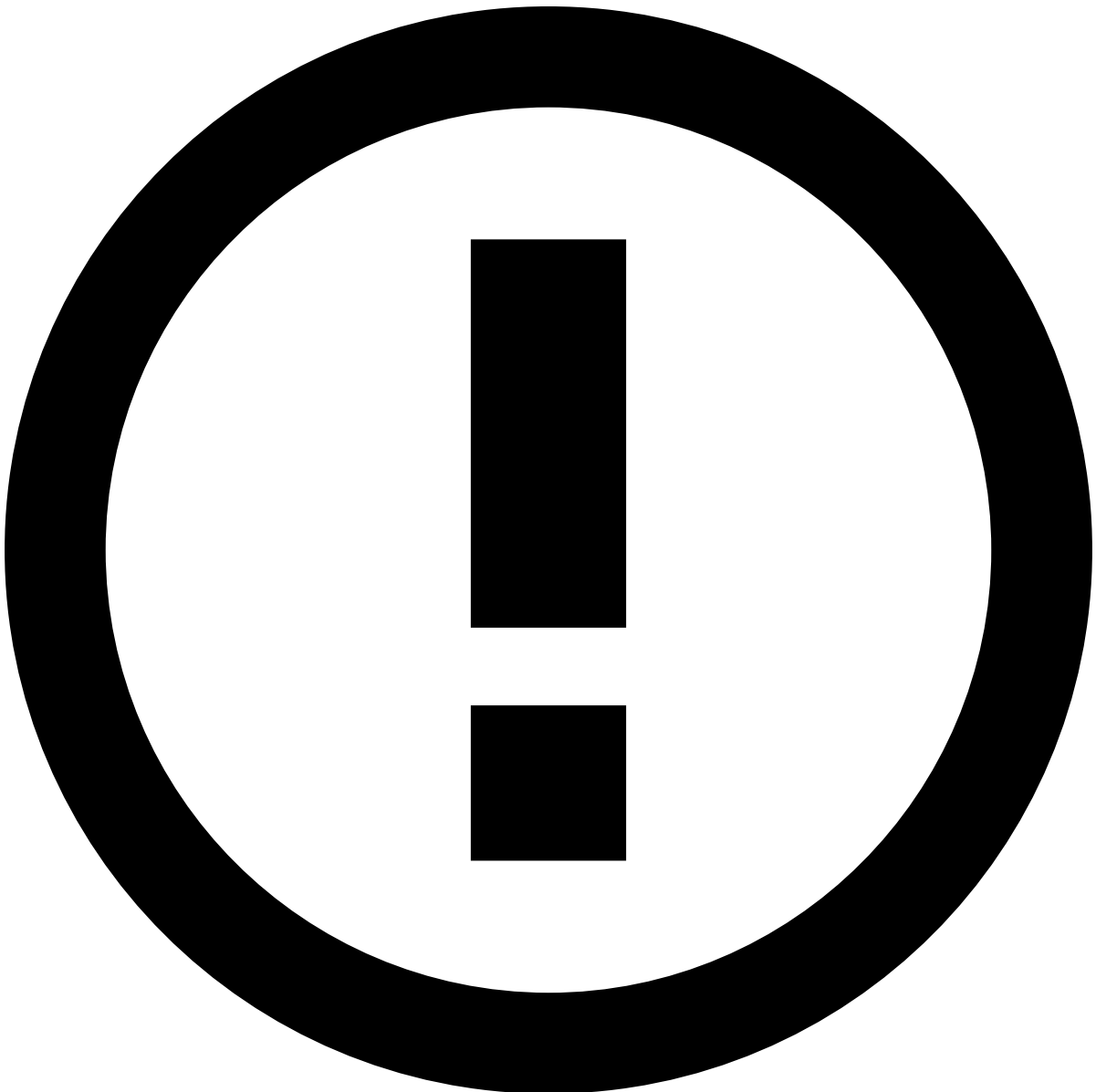
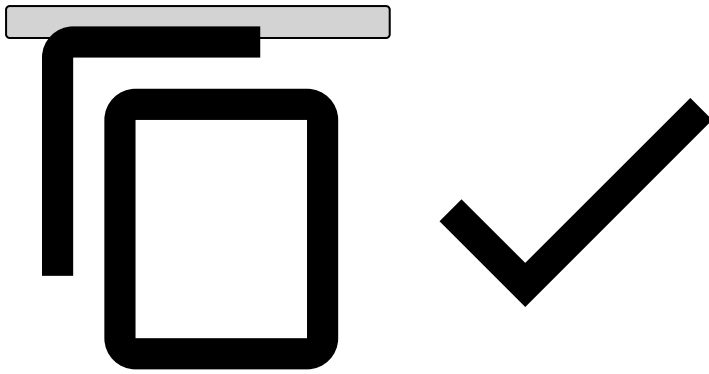
When it is necessary to correlate multiple transactions, try to write the rule by looking at past transactions instead of future ones. Instead of "The next transaction will be in the same location", think "The previous transaction was on the same location".

Since windows are just lists, it is possible to use the collection methods:

Example 4: match transactions that surpass a \$1000/hour limit

```
// Variables
// Get the last 1 hour of transactions for the current card number
last1hEvents = event[1 hour by card_number];

// Condition
last1hEvents.sum(amount) > 1000
```

Methods of Collections in RDL

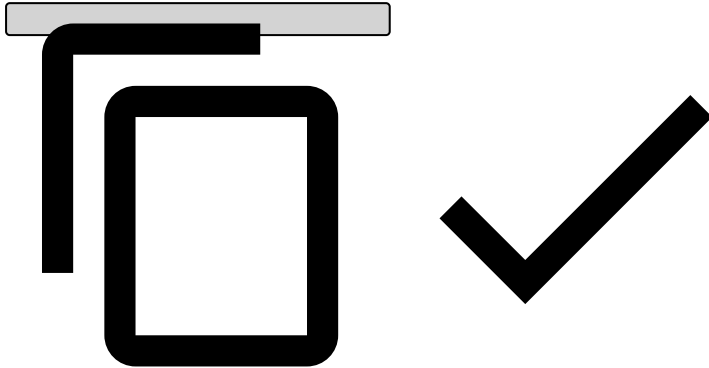
Collections in RDL have several methods, including `all` , `any` , `contains` , `filter` , `count` , etc.

It is also possible to use lambdas:

Example 5: match 5 sequential transactions of the same card that have a low amount (< \$5)

```
// Variables
last5Events = event[5 tuples by card_number];

// Condition
last5Events.count() == 5 and lastEvents.all(e => e.amount < 5)
```





Lambdas Can Impact Performance Severely

Try to only use lambdas for collections whose size is known to be small (<100 elements) and prefer filtering in the window (using the **where** operator) to the filter method.

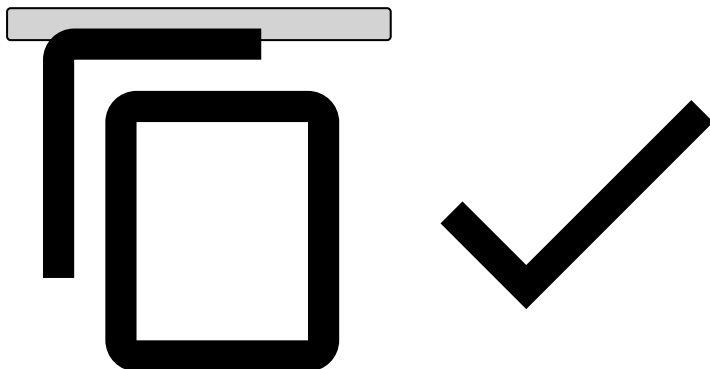
Using Templates to Reuse Code

Rule Templates allow to parameterize rules and as such are useful when it is necessary to have several instances of the same logic but with different parameters.

One example would be writing a rule to trigger an alert if the sum of amount of the last N minutes of a specific card and terminal type is more than Y. If you needed to write only one or two rules with this logic it would be perfectly acceptable to write it like:

```
// Variables
lastEvents = event[10 minutes where terminaltype == &quot;ATM&quot; by cardnumber];

// Condition
lastEvents.sum(amount) &gt; 1000
```

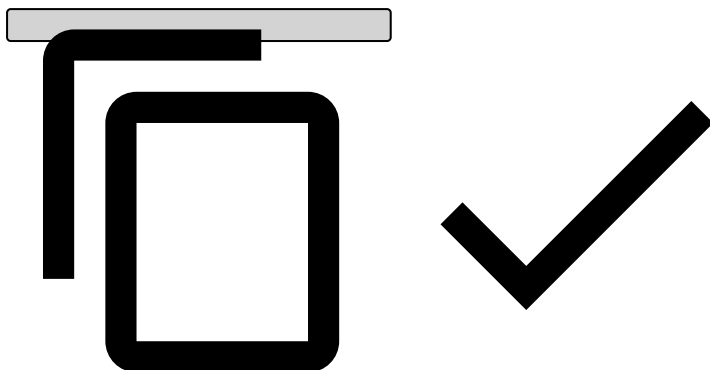


However repeating this several times is error prone and cumbersome.

Using templates, we replace the literal values by template parameters:

```
// Variables
lastEvents = event[$nMin minutes where terminaltype == $terminaltype by cardnumber];

// Condition
lastEvents.sum(amount) &gt; $yDolars
```



Then to create rules we select the template and fill in the specific values:

Figure 6. Reusing RDL Code with Templates

Detect Repeated Transaction Attempts With the Same Location and Amount

A common pattern for fraudulent transactions is to make several transaction attempts in the same location or change the location after several attempts. For example:

- Transaction 1, Location = Lisbon, amount = 50: accept
- Transaction 2, Location = Lisbon, amount = 50: accept
- Transaction 3, Location = Lisbon, amount = 50: accept
- Transaction 4, Location = Lisbon, amount = 50: accept
- Transaction 5, Location = Lisbon, amount = 50: block

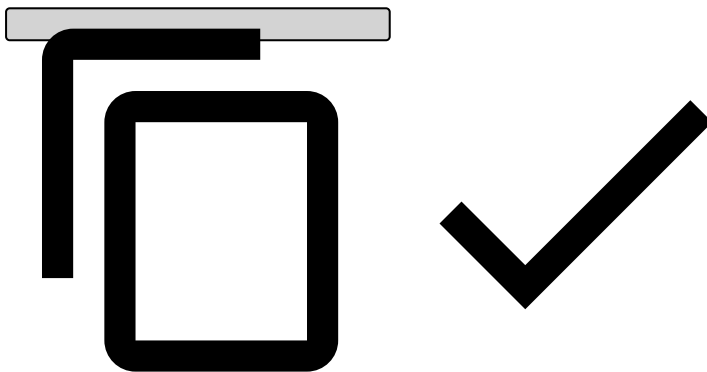
Or:

- Transaction 1, Location = Lisbon, amount = 50: accept
- Transaction 2, Location = Lisbon, amount = 50: accept
- Transaction 3, Location = Lisbon, amount = 50: accept
- Transaction 4, Location = Lisbon, amount = 50: accept
- Transaction 5, Location = Coimbra, amount = 50: block

The following RDL code detects such patterns by first checking if either all or just the first 4 events have the same location ($ev.location == first.location$) and secondly by making sure that that *all* lambda always evaluates to true for the last event ($ev.uuid == event.uuid$):

```
// Variables
lastEvents = event[5 tuples by card];
first = lastEvents.at(1);

// Condition
lastEvents.count() == 5 and
  lastEvents.all(ev.location == first.location or ev.uuid == event.uuid)
```



Detect Transactions With a Considerable Amount After a 1\$ Transaction in the Same Location

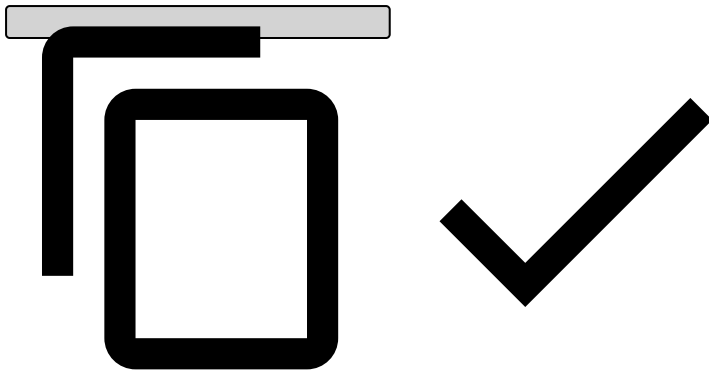
A common pattern for fraudulent transactions is to make an initial test transaction with a low amount and then make transaction with a large amount. For example:

- Transaction 1, Location = Lisbon, Amount = 1: accept
- Transaction 2, Location = Lisbon, Amount = 200: block

The following RDL detects such a pattern by picking the last two events (for the current event card number) and checking if the location is the same in the two and if the respective amounts are in the ranges described before:

```
// Variables
lastEvents = event[2 tuples by card];
first = lastEvents.at(1);

// Condition
event.amount > 100 and lastEvents.count() == 2 and
  first.amount <= 1 and
  first.location == event.location
```



Detect Repeated Transaction Attempts With Decreasing Amounts [🔗](#)

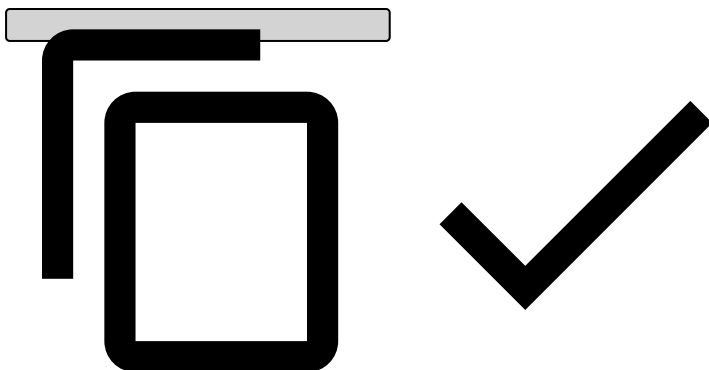
A common pattern for fraudulent transactions is to try to make a transaction with a large amount, decrease it if it fails (usually by dividing the amount by 2 or 3) and try again. For example:

- Transaction 1, amount = 1000: accept
- Transaction 2, amount = 500: accept
- Transaction 3, amount = 250: block

The following RDL detects such a pattern for a multiple of 2 by asserting if the first 2 events of the last 3 events (if there are 3 events in the window) are multiples of the current amount.

```
// Variables
lastEvents = event[3 tuples by card];

// Condition
lastEvents.count() == 3 and
lastEvents.at(1) == event.amount * 4 and
lastEvents.at(2) == event.amount * 2
```



Detect Bursts of Transaction Attempts and Alert When the Transaction Comes Through [🔗](#)

A common pattern for fraudulent transactions is to make attempts at performing a given operation (e.g., buying some goods) until the transaction is accepted (trying a different CCV at each attempt).

Suppose that code = 5 means that the transaction failed and code = 10 means that the transaction was accepted.

One example of this pattern could be:

- Transaction 1, Location = Lisbon, code = 5: no alert
- Transaction 2, Location = Lisbon, code = 5: no alert
- Transaction 3, Location = Lisbon, code = 10: alert

The following RDL detects such a pattern by first checking if the location is the same in all the transactions (to determine that the origin is the same) and then by checking that all the transactions have code 5 (failed) but the last (the current) which was accepted:

```
// Variables
lastEvents = event[3 tuples by card];
firstEvent = lastEvents.at(1);

// Condition
lastEvents.count() == 3 and
lastEvents.all(
  ev => ev.location == firstEvent.location and
  (ev.code == 5 or (ev.uuid == event.uuid and ev.code == 10))
)
```

